Lab Sheet 07

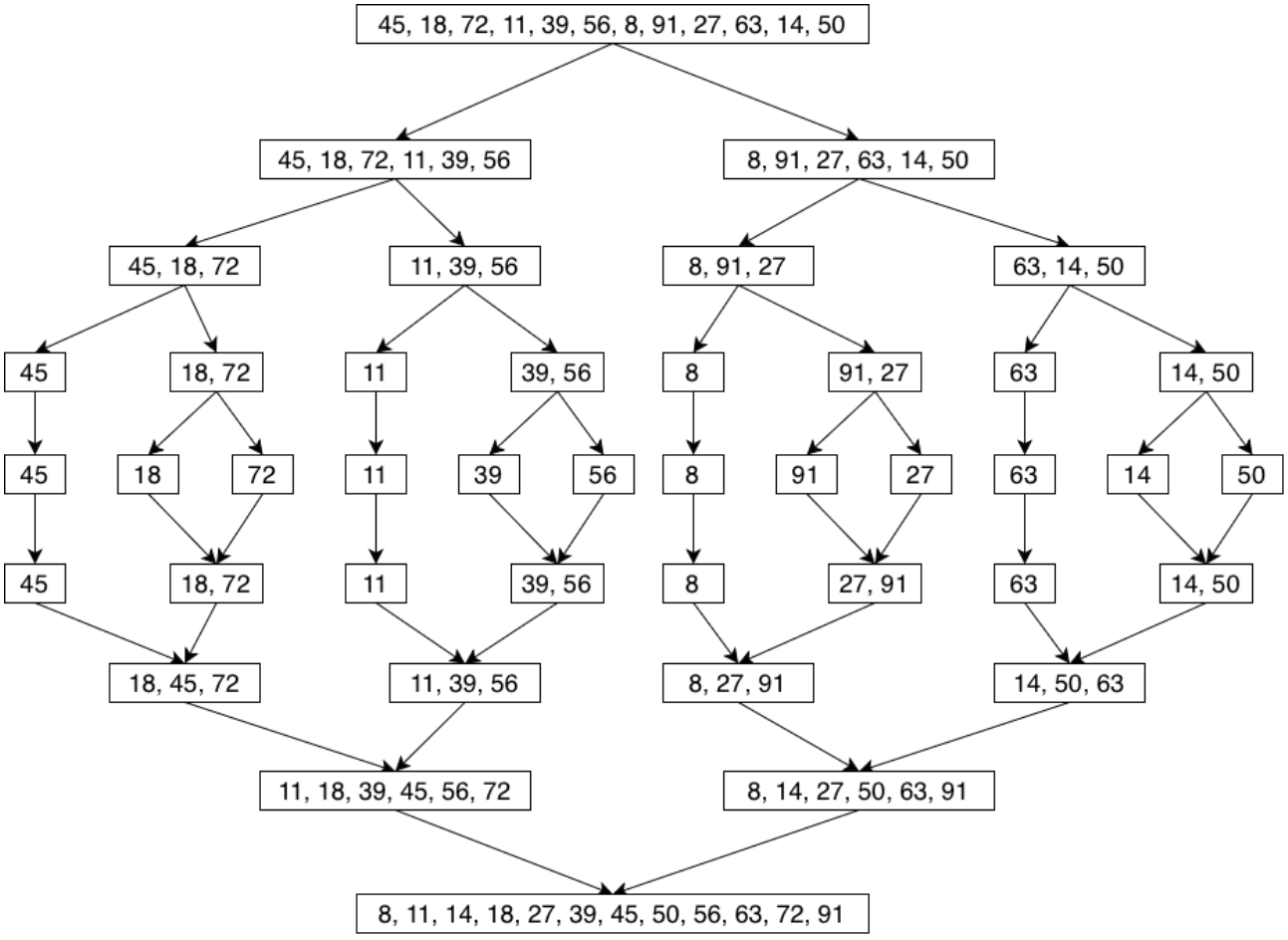
Registration No. : 2025/CS/079
Index No. : 25000799

(01)

A.

Step	Phase	Input	Output
1	Split	[45, 18, 72, 11, 39, 56, 8, 91, 27, 63, 14, 50]	[45, 18, 72, 11, 39, 56] [8, 91, 27, 63, 14, 50]
2	Split	[45, 18, 72, 11, 39, 56] [8, 91, 27, 63, 14, 50]	[45, 18, 72] [11, 39, 56] [8, 91, 27] [63, 14, 50]
3	Split	[45, 18, 72] [11, 39, 56] [8, 91, 27] [63, 14, 50]	[45] [18, 72] [11] [39, 56] [8] [91, 27] [63] [14, 50]
4	Split	[45] [18, 72] [11] [39, 56] [8] [91, 27] [63] [14, 50]	[45] [18] [72] [11] [39] [56] [8] [9] [27] [63] [14] [50]
5	Merge	[45] [18] [72] [11] [39] [56] [8] [9] [27] [63] [14] [50]	[45] [18, 72] [11] [39, 56] [8] [27, 91] [63] [14, 50]
6	Merge	[45] [18, 72] [11] [39, 56] [8] [27, 91] [63] [14, 50]	[18, 45, 72] [11, 39, 56] [8, 27, 91] [14, 50, 63]
7	Merge	[18, 45, 72] [11, 39, 56] [8, 27, 91] [14, 50, 63]	[11, 18, 39, 45, 56, 72] [8, 14, 27, 50, 63, 91]
8	Merge	[11, 18, 39, 45, 56, 72] [8, 14, 27, 50, 63, 91]	[8, 11, 14, 18, 27, 39, 45, 50, 56, 63, 72, 91]

B.



C.

1 :

Six individual elements. So 6 comparisons.

2 :

Six arrays with two elements. Three comparisons between each array and three comparisons between the elements.

Therefore 9 comparisons

3 :

Two arrays each with 6 elements. 16 comparisons are required to sort the array completely.

Total comparisons = 6 + 9 + 16 = 31

D.

Best case scenario:

Array is already sorted.

Time complexity = O(n log n)

Average case scenario:

Array is randomly sorted.

Time complexity = O(n log n)

Worst case scenario:

Array is sorted in the opposite order.

Time complexity = O(n log n)

Merge sort always divides the array to sort it. Even if the array is sorted, it will keep dividing and comparing the elements. Therefore regardless of the order the time complexity remains the same.

(02)

A.

The Brute Force method checks every possible contiguous subarray, computes its sum, and keeps track of the maximum found so far.

Steps:

- Use two nested loops: outer loop fixes the starting index i, inner loop fixes the ending index j (with $j \geq i$).
- For each pair (i, j), compute the sum of elements from i to j.
- Compare this sum with the current maximum; update the maximum if the new sum is larger.
- After checking all pairs, the maximum recorded value is the answer.

Time complexity:

- If the sum for each subarray is recomputed from scratch, this is $O(n^3)$.
- If a running sum is kept while extending j, this improves to $O(n^2)$.

B.

Subarray	Sum
[250]	250
[250, -120]	130
[250, -120, 340]	470
[250, -120, 340, -180]	290
[250, -120, 340, -180, 520]	810
[250, -120, 340, -180, 520, -90]	720
[250, -120, 340, -180, 520, -90, 160]	880
[250, -120, 340, -180, 520, -90, 160, -240]	640
[250, -120, 340, -180, 520, -90, 160, -240, 430]	1070
[250, -120, 340, -180, 520, -90, 160, -240, 430, -110]	960
[-120]	-120
[-120, 340]	220
[-120, 340, -180]	40
[-120, 340, -180, 520]	560
[-120, 340, -180, 520, -90]	470
[-120, 340, -180, 520, -90, 160]	630
[-120, 340, -180, 520, -90, 160, -240]	390
[-120, 340, -180, 520, -90, 160, -240, 430]	820
[-120, 340, -180, 520, -90, 160, -240, 430, -110]	710
[340]	340
[340, -180]	160
[340, -180, 520]	680
[340, -180, 520, -90]	590
[340, -180, 520, -90, 160]	750
[340, -180, 520, -90, 160, -240]	510
[340, -180, 520, -90, 160, -240, 430]	940
[340, -180, 520, -90, 160, -240, 430, -110]	830
[-180]	-180
[-180, 520]	340
[-180, 520, -90]	250

Subarray	Sum
[-180, 520, -90, 160]	410
[-180, 520, -90, 160, -240]	170
[-180, 520, -90, 160, -240, 430]	600
[-180, 520, -90, 160, -240, 430, -110]	490
[520]	520
[520, -90]	430
[520, -90, 160]	590
[520, -90, 160, -240]	350
[520, -90, 160, -240, 430]	780
[520, -90, 160, -240, 430, -110]	670
[-90]	-90
[-90, 160]	70
[-90, 160, -240]	-170
[-90, 160, -240, 430]	260
[-90, 160, -240, 430, -110]	150
[160]	160
[160, -240]	-80
[160, -240, 430]	350
[160, -240, 430, -110]	240
[-240]	-240
[-240, 430]	190
[-240, 430, -110]	80
[430]	430
[430, -110]	320
[-110]	-110

[250, -120, 340, -180, 520, -90, 160, -240, 430]
Maximum sum = 1070

C.

Total subarrays to examine = n(n+1)/2 = 10×11/2 = 55
Total comparisons = 55 - 1 = 54

D.

Brute Force is $O(n^2)$ or $O(n^3)$. As n grows, the number of subarrays grows quadratically and the work per subarray also scales so runtime blows up fast for large datasets.

A much more efficient option is Kadane's Algorithm, which solves the same problem in O(n) time using a single pass. The Divide and Conquer approach is another improvement, running in O(n log n).

(03)

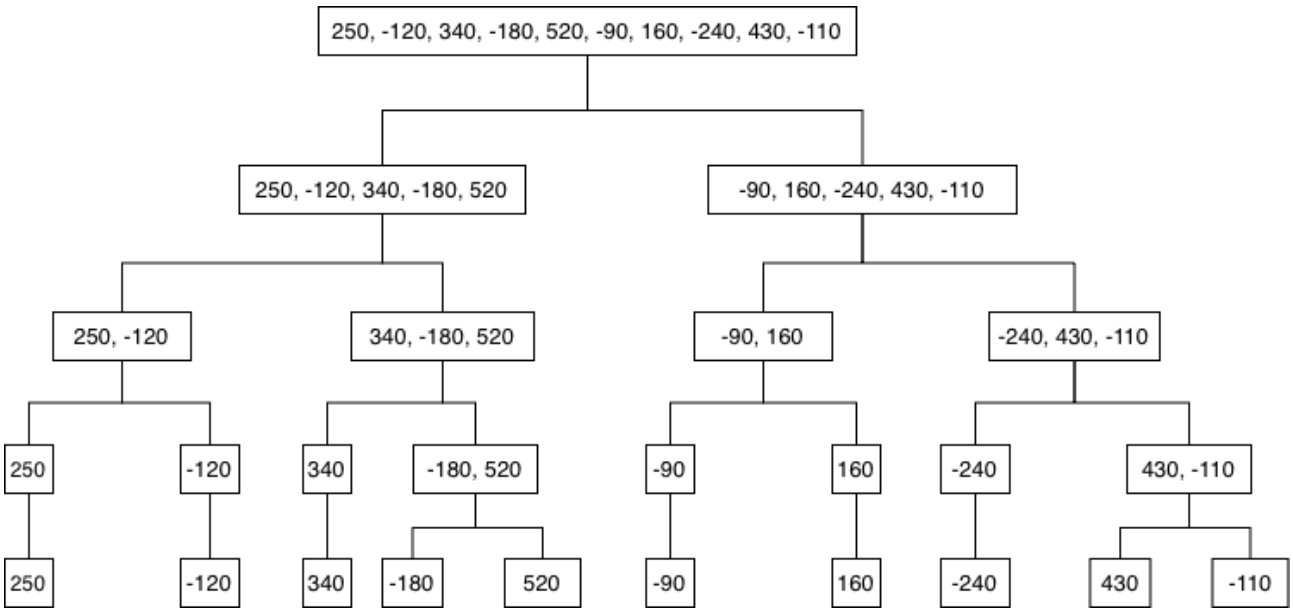
A.

The Divide and Conquer algorithm solves this problem by recursively breaking down the array:

Divide: Split the array into two halves at the midpoint.

Conquer: Recursively find the maximum subarray sum in the left half and the right half.
260 +
Combine: Find the maximum subarray sum that crosses the midpoint. The final result is the maximum of the three values: (Max left sum, Max right sum, Max crossing sum).

B.



C.

Level 1 :

Left half = 250 I Right half = 250 I Crossing subarray = 250
Left half = -120 I Right half = -120 I Crossing subarray = -120

Left half = 340 I Right half = 340 I Crossing subarray = 340
Left half = -180 I Right half = -180 I Crossing subarray = -180
Left half = 520 I Right half = 520 I Crossing subarray = 520

Left half = -90 I Right half = -90 I Crossing subarray = -90
Left half = 160 I Right half = 160 I Crossing subarray = 160

Left half = -240 I Right half = -240 I Crossing subarray = -240
Left half = 430 I Right half = 430 I Crossing subarray = 430
Left half = -110 I Right half = -110 I Crossing subarray = -110

Level 2 :

Left half = 250 | Right half = 250 | Crossing subarray = 250
Left half = -120 | Right half = -120 | Crossing subarray = -120

Left half = 340 | Right half = 340 | Crossing subarray = 340
Left half = -180 | Right half = 520 | Crossing subarray = 340

Left half = -90 | Right half = -90 | Crossing subarray = -90
Left half = 160 | Right half = 160 | Crossing subarray = 160

Left half = -240 | Right half = -240 | Crossing subarray = -240
Left half = 430 | Right half = -110 | Crossing subarray = 320

Level 3 :

Left half = 250 | Right half = -120 | Crossing subarray = 130
Left half = 340 | Right half = 520 | Crossing subarray = 680

Left half = -90 | Right half = 160 | Crossing subarray = 70
Left half = -240 | Right half = 430 | Crossing subarray = 190

Level 4 :

Left half = 250 | Right half = 680 | Crossing subarray = 810
Left half = 160 | Right half = 430 | Crossing subarray = 350

Level 5 :

Left half = 810 | Right half = 430 | Crossing subarray = 1070

Maximum sub-array sum = 1070

D.

The recurrence relation in divide and conquer approach, results in O(nlogn).
It is more efficient than Brute Force ($O(n^2)$) for large datasets because it reduces the number of subproblems processed by using a logarithmic divide-and-conquer structure rather than nested loops.

E.

Feature	Brute Force	Divide and Conquer
Main Idea	Check every possible subarray	Recursive splitting and merging
Time complecity	$O(n^2)$	O(nlogn)
Efficiency	Efficient only for smaller data sets	High efficient for larger data sets while recursion overheads may occur in smaller data sets
Advantages / limitaions	Slow growth rate and poor scalability	More complex to implement